

Laboratório 6 - Relatório

Nícolas Rosenthal Dal Corso

20 de maio de 2026

Sumário

1	Introdução	2
2	Estrutura Geral do Pipeline	2
2.1	Pré-processamento em R	2
2.2	Modelagem em Python	3
3	Pré-processamento e Engenharia de Features	3
3.1	Imputação de Valores Ausentes	3
3.1.1	CryoSleep	3
3.1.2	VIP	3
3.1.3	Gastos	4
3.2	Engenharia de Features	4
3.2.1	Expenditure e NoExpenditure	4
3.2.2	Cabin <i>Features</i>	4
3.2.3	Passenger Groups	4
3.3	Transformações Numéricas	4
4	Pipeline de Modelagem	5
4.1	Estratégia Geral	5
5	Encoding Categórico	5
6	Modelos Utilizados	5
6.1	CatBoost	5
6.2	LightGBM	5
6.3	XGBoost	6

6.4	ExtraTrees	6
7	Otimização Bayesiana com Optuna	6
8	Validação Cruzada	6
9	<i>Ensemble Stacking</i>	6
10	Otimização de <i>Threshold</i>	7
11	Métricas Utilizadas	7
11.1	Accuracy	7
11.2	F1 Score	7
11.3	ROC AUC	7
12	Resultados	7

1 Introdução

Este relatório descreve o pipeline completo desenvolvido para o problema **Spaceship Titanic**, da plataforma Kaggle.

O objetivo da competição consiste em prever se um passageiro foi transportado para outra dimensão, representado pelo predicado

Transported {True, False}

Partindo da análise exploratória de dados e da engenharia de *features* desenvolvida anteriormente, o pipeline implementado utiliza *ensemble stacking* e otimização Bayesiana com Optuna; validação cruzada estratificada, otimização de *threshold*, e modelos *gradient boosting* modernos (CatBoost, LightGBM, XGBoost)

O foco principal do projeto foi maximizar desempenho preditivo, robustez estatística e capacidade de generalização.

2 Estrutura Geral do Pipeline

O pipeline foi dividido em duas etapas principais:

2.1 Pré-processamento em R

- EDA (*Exploratory Data Analysis*);

- imputação de valores ausentes;
- transformação de variáveis;
- feature engineering;
- normalização estrutural;
- exportação dos *datasets* processados.

Arquivos gerados:

data/processed/train_processed.csv

data/processed/test_processed.csv

2.2 Modelagem em Python

- *encoding* categórico;
- otimização de hiperparâmetros;
- treinamento dos modelos;
- *stacking ensemble*;
- avaliação;
- geração da submissão final.

3 Pré-processamento e Engenharia de Features

3.1 Imputação de Valores Ausentes

Os valores ausentes foram tratados em R utilizando regras heurísticas baseadas na análise exploratória.

3.1.1 CryoSleep

A *feature* CryoSleep foi imputada utilizando a relação observada entre **ausência de gastos** e **estado criogênico**.

3.1.2 VIP

A variável VIP foi imputada utilizando distribuição empírica baseada em níveis elevados de gastos totais.

3.1.3 Gastos

Passageiros em criossono tiveram os gastos redefinidos para zero:

- RoomService
- FoodCourt
- ShoppingMall
- Spa
- VRDeck

3.2 Engenharia de Features

3.2.1 Expenditure e NoExpenditure

Soma total dos gastos do passageiro:

```
Expenditure =  
RoomService +  
FoodCourt +  
ShoppingMall +  
Spa +  
VRDeck
```

3.2.2 Cabin *Features*

A variável Cabin foi decomposta em Cabin_Deck, Cabin_Number, Cabin_Side

3.2.3 Passenger Groups

A partir de PassengerId foi criada a variável Group permitindo identificar passageiros viajando em conjunto.

3.3 Transformações Numéricas

Foram avaliadas múltiplas transformações `log1p`, `sqrt`, `square` e `Yeo-Johnson`. As melhores transformações foram selecionadas via validação cruzada estratificada.

4 Pipeline de Modelagem

4.1 Estratégia Geral

O modelo final utiliza *stacking ensemble* segundo a arquitetura:

Base Models

Probabilidades Out-Of-Fold

Meta Learner (Logistic Regression)

Predição Final

5 Encoding Categórico

As variáveis categóricas foram codificadas utilizando *OrdinalEncoder*

6 Modelos Utilizados

6.1 CatBoost

Modelo baseado em *gradient boosting* com suporte nativo a variáveis categóricas.

Características:

- excelente desempenho tabular;
- robustez contra *overfitting*;
- alta capacidade de modelagem não-linear.

6.2 LightGBM

Gradient boosting baseado em árvores *histogram-based*.

Características:

- treinamento extremamente rápido;
- alta escalabilidade;
- excelente desempenho em *datasets* tabulares.

6.3 XGBoost

Framework clássico de *gradient boosting*.

Características:

- regularização forte;
- alta estabilidade;
- ótimo desempenho em competições Kaggle.

6.4 ExtraTrees

Modelo *ensemble* baseado em árvores extremamente aleatórias.

Características:

- redução de variância;
- complementaridade em relação aos modelos boosting.

7 Otimização Bayesiana com Optuna

Os hiperparâmetros foram ajustados utilizando `Optuna` (*Bayesian Optimization* + *Tree-structured Parzen Estimator*, *tree pruning* e busca adaptativa de hiperparâmetros).

8 Validação Cruzada

Foi utilizada *Stratified K-Fold Cross Validation* com $k = 10$ para reduzir variância, preservar distribuição da classe alvo e estimar generalização real do modelo.

9 Ensemble Stacking

O ensemble final foi construído utilizando *StackingClassifier*

CatBoost

LightGBM

XGBoost

ExtraTrees

Logistic Regression

O meta-modelo aprende a combinar as probabilidades dos modelos base.

10 Otimização de *Threshold*

Ao invés do threshold padrão 0.5, foi realizada busca exaustiva no intervalo [0.30, 0.70] para maximizar acurácia OOF e melhorar calibração das classes.

11 Métricas Utilizadas

11.1 Accuracy

Accuracy =
Predições corretas / Total

Métrica principal da competição: RMSE.

11.2 F1 Score

Balanceia precisão e recall. Importante em casos de desbalanceamento.

11.3 ROC AUC

Mede capacidade discriminativa probabilística.

12 Resultados

O pipeline desenvolvido tipicamente obtém:

Modelo	Acurácia Esperada
CatBoost simples	[0.79, 0.81]
Ensemble básico	[0.81, 0.82]
Stack + Optuna	[0.82, 0.84]